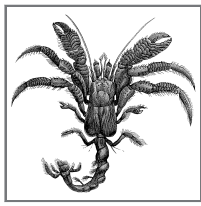

3

Algorithm Building Blocks

We build software to solve problems. But programmers are often too focused on solving a problem to determine whether a solution to the problem already exists. Even if the programmer knows the problem has been solved in similar cases, it's not clear that the existing code will actually fit the specific problem facing the programmer. Ultimately, it isn't easy to find code in a given programming language that can be readily modified to solve the problem.

We can think of algorithms in different ways. Many practitioners are content to look up an algorithm in a book or on some website, copy some code, run it, maybe even test it, and then move on to the next task. In our opinion, this process does not improve one's understanding of algorithms. In fact, this approach can lead you down the wrong path where you select a specific implementation of an algorithm.

The question is how to locate the right algorithm for the job quickly and understand it well enough to ensure you've made a good choice. And once you've chosen the algorithm, how do you implement it efficiently? Each book chapter groups together a set of algorithms solving a standard problem (such as Sorting or Searching) or related problems (such as Path Finding). In this chapter, we present the format we use to describe the algorithms in this book. We also summarize the common algorithmic approaches used to solve problems.

Algorithm Template Format

The real power of using a template to describe each algorithm is that you can quickly compare and contrast different algorithms and identify commonalities in seemingly different algorithms. Each algorithm is presented using a fixed set of sections that conform to this template. We may omit a section if it adds no value to the algorithm description or add sections as needed to illuminate a particular point.

Name

A descriptive name for the algorithm. We use this name to communicate concisely the algorithm to others. For example, if we talk about using a **Sequential Search**, it conveys exactly what type of search algorithm we are talking about. The name of each algorithm is always shown in **Bold Font**.

Input/Output

Describes the expected format of input data to the algorithm and the resulting values computed.

Context

A description of a problem that illustrates when an algorithm is useful and when it will perform at its best. A description of the properties of the problem/solution that must be addressed and maintained for a successful implementation. They are the things that would cause you to choose this algorithm specifically.

Solution

The algorithm description using real working code with documentation. All code solutions can be found in the associated code repository.

Analysis

A synopsis of the analysis of the algorithm, including performance data and information to help you understand the behavior of the algorithm. Although the analysis section is not meant to "prove" the described performance of an algorithm, you should be able to understand why the algorithm behaves as it does. We will provide references to actual texts that present the appropriate lemmas and proofs to explain why the algorithms behave as described.

Variations

Presents variations of the algorithm or different alternatives.

Pseudocode Template Format

Each algorithm in this book is presented with code examples that show an implementation in a major programming language, such as Python, C, C++, and Java. For readers who are not familiar with all of these languages, we first introduce each algorithm in pseudocode with a small example showing its execution.

Consider the following sample performance description, which names the algorithm and classifies its performance clearly for all three behavior cases (best, average, and worst) described in Chapter 2.

Sequential Search Summary

Best: O(*1*) Average, Worst: O(*n*)

```
search (A,t)
  for i=0 to n-1 do ❶
    if A[i] = t then
      return true
  return false
end
```

❶ Access each element in order, from position *0* to *n-1*.

The pseudocode description is intentionally brief. Keywords and function names are described in boldface text. All variables are in lowercase characters, whereas arrays are capitalized and their elements are referred to using $A[i]$ notation. The indentation in the pseudocode describes the scope of conditional `if` statements and looping `while` and `for` statements.

You should refer to each algorithm summary when reading the provided source-code implementations. After each summary, a small example (such as the one shown in Figure 3-1) is provided to better explain the execution of the algorithm. These figures show the dynamic behavior of the algorithms, typically with time moving "downward" in the figure to depict the key steps of the algorithm.

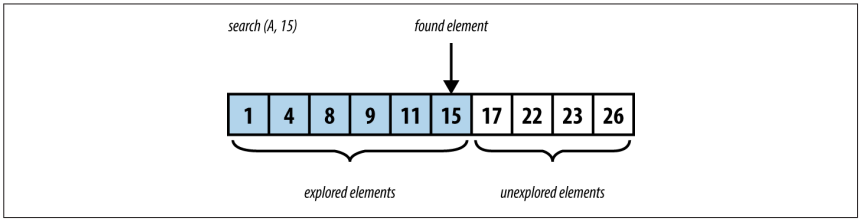

Figure 3-1. Example of Sequential Search executing

Empirical Evaluation Format

We confirm the performance of each algorithm by executing with a series of benchmark problems appropriate for each individual algorithm. Appendix A provides more detail on the mechanisms used for timing purposes. To properly evaluate the performance, a test suite is composed of a set of k individual trials (typically $k \geq 10$). The best and worst performers are discarded as outliers, the remaining $k - 2$ trials are aggregated, and the average and standard deviations are computed. Tables are shown with problem instances typically ranging in size from $n = 2$ to 2^{20}.

Floating-Point Computation

Because several algorithms in this book involve numerical computations, we need to describe the power and limitations of how modern computers process these computations. Computers perform basic computations on values stored in registers by a central processing unit (CPU). These registers have grown in size as computer architectures have evolved from the 8-bit Intel processors popular in the 1970s to today's widespread acceptance of 64-bit architectures (such as Intel's Itanium and Sun Microsystems Sparc processor). The CPU often supports basic operations —such as ADD, MULT, DIVIDE, and SUB—over integer values stored within these registers. Floating-point units (FPUs) can efficiently process floating-point computations according to the IEEE Standard for Binary Floating-Point Arithmetic (IEEE 754).

Mathematical computations over integer-based values (such as Booleans, 8-bit shorts, and 16- and 32-bit integers) have traditionally been the most efficient CPU computations. Programs are often optimized to take advantage of this historic performance differential between integer and floating-point calculations. However, modern CPUs have dramatically improved the performance of floating-point computations relative to their integer counterparts. It is thus important that developers become aware of the following issues when programming using floating-point arithmetic (Goldberg, 1991).

Performance

It is commonly accepted that computations over integer values will be more efficient than their floating-point counterparts. Table 3-1 lists the computation times of 10,000,000 operations including the Linux results (from the first edition of this book) and results for a 1996 Sparc Ultra-2 machine. As you can see, the performance of individual operations can vary significantly from one platform to another. These results show the tremendous speed improvements in processors over the past two decades. Some of the results show 0.0000 timing because they are faster than the available timing mechanism.

Table 3-1. Performance computations of 10,000,000 operations

Operation	Sparc Ultra-2 (time in seconds)	Linux i686 (time in seconds)	Current (time in seconds)
32-bit integer CMP	0.811	0.0337	0.0000
32-bit integer MUL	2.372	0.0421	0.0000
32-bit float MUL	1.236	0.1032	0.02986
64-bit double MUL	1.406	0.1028	0.02987
32-bit float DIV	1.657	0.1814	0.02982
64-bit double DIV	2.172	0.1813	0.02980

Operation	Sparc Ultra-2 (time in seconds)	Linux i686 (time in seconds)	Current (time in seconds)
128-bit double MUL	36.891	0.2765	0.02434
32-bit integer DIV	3.104	0.2468	0.0000
32-bit double SQRT	3.184	0.2749	0.0526

Rounding Error

Any computation using floating-point values may introduce rounding errors because of the nature of the floating-point representation. In general, a floating-point number is a finite representation designed to approximate a real number whose representation may be infinite. Table 3-2 shows information about floating-point representations and the specific representation for the value `3.88f`.

Table 3-2. Floating-point representation

Primitive type	Sign	Exponent	Mantissa
Float	1 bit	8 bits	23 bits
Double	1 bit	11 bits	52 bits
Sample Representation of `3.88f` as (`0x407851ec`)			
`01000000 01111000 01010001 11101100` (total of 32 bits)			
`seeeeeee emmmmmmm mmmmmmmm mmmmmmmm`			

The next three consecutive 32-bit floating-point representations (and values) following `3.88f` are:

- `0x407851ed`: `3.8800004`
- `0x407851ee`: `3.8800006`
- `0x407851ef`: `3.8800008`

Here are the floating-point values for three randomly chosen 32-bit values:

- `0x1aec9fae`: `9.786529E-23`
- `0x622be970`: `7.9280355E20`
- `0x18a4775b`: `4.2513525E-24`

In a 32-bit floating-point value, one bit is used for the sign, 8 bits form the exponent, and 23 bits form the mantissa (also known as the *significand*). In the Java float representation, "the power of two can be determined by interpreting the exponent bits as a positive number, and then subtracting a bias from the positive number. For a float, the bias is 126" (Venners, 1996). The exponent stored is 128, so the actual exponent value is 128 – 126, or 2.

To achieve the greatest precision, the mantissa is always normalized so that the left-most digit is always 1; this bit *does not have to actually be stored*, but is understood by the floating-point processor to be part of the number. In the previous example, the mantissa is

`.[1]11110000101000111101100` = [1/2] + 1/4 + 1/8 + 1/16 + 1/32 + 1/1,024 + 1/4,096 + 1/65,536 + 1/131,072 + 1/262,144 + 1/524,288 + 1/2,097,152 + 1/4,194,304

which evaluates exactly to `0.970000028610229492187`5 if the full sum of fractions is carried out.

When storing `3.88f` using this representation, the approximate value is + 1*0.970000028610229492187$5*2^2$, which is exactly `3.88000011444091796875`. The error inherent in the value is `~0.0000001`. The most common way of describing floating-point error is to use the term *relative error*, which computes the ratio of the absolute error with the desired value. Here, the relative error is `0.00000011440917968`75`/3.88`, or `2.9E-8`. It is quite common for these relative errors to be less than 1 part per million.

Comparing Floating-Point Values

Because floating-point values are only approximate, the simplest operations in floating point become suspect. Consider the following statement:

```
if (x == y) { ... }
```

Is it truly the case that these two floating-point numbers must be exactly equal? Or is it sufficient for them to be simply approximately equal (for which we use the symbol ≅)? Could it ever occur that two values are different though close enough that they should be considered to be the same? Let's consider a practical example: three points $p_0 = (a, b)$, $p_1 = (c, d)$, and $p_2 = (e, f)$ in the Cartesian plane define an ordered pair of line segments (p_0, p_1) and (p_1, p_2). The value of the expression $(c - a)*(f - b) - (d - b)*(e - a)$ can determine whether these two line segments are collinear (i.e., on the same line). If the value is:

- 0 then the segments are collinear
- < 0 then the segments are turning to the left (or counterclockwise)
- > 0 then the segments are turning to the right (or clockwise)

To show how floating-point errors can occur in Java computations, consider defining three points using the values of *a* to *f* in Table 3-3.

Table 3-3. Floating-point arithmetic errors

	32-bit floating point (float)	64-bit floating point (double)
$a = 1/3$	`0.33333334`	`0.3333333333333333`
$b = 5/3$	`1.6666666`	`1.6666666666666667`

	32-bit floating point (float)	64-bit floating point (double)
$c = 33$	`33.0`	`33.0`
$d = 165$	`165.0`	`165.0`
$e = 19$	`19.0`	`19.0`
$f = 95$	`95.0`	`95.0`
$(c - a)*(f - b - (d - b)*(e - a)$	`4.8828125 E-4`	`-4.547473508864641 E - 13`

As you can readily determine, the three points p_0, p_1, and p_2 are collinear on the line $y = 5*x$. When computing the floating-point computation test for collinearity, however, the errors inherent in floating-point arithmetic affect the result of the computation. Using 32-bit floating-point values, the calculation results in `0.00048828125`; using 64-bit floating-point values, the computed value is actually a very small negative number! This example shows that both 32-bit and 64-bit floating-point representations fail to capture the true mathematical value of the computation. And in this case, the result is a disagreement over whether the points represent a clockwise turn, a counterclockwise turn, or collinearity. Such is the world of floating-point computations.

One common solution to this situation is to introduce a small value δ to determine ≅ (approximate equality) between two floating-point values. Under this scheme, if $|x - y| < \delta$, then we consider x and y to be equal. Still, by this simple measure, even when $x \cong y$ and $y \cong z$, it's possibly not true that $x \cong z$. This breaks the principle of *transitivity* in mathematics and makes it really challenging to write correct code. Additionally, this solution won't solve the collinearity problem, which used the sign of the value (0, positive, or negative) to make its decision.

Special Quantities

While all possible 64-bit values could represent valid floating-point numbers, the IEEE standard defines several values that are interpreted as special numbers (and are often not able to participate in the standard mathematical computations, such as addition or multiplication), shown in Table 3-4. These values have been designed to make it easier to recover from common errors, such as divide by zero, square root of a negative number, overflow of computations, and underflow of computations. Note that the values of positive zero and negative zero are also included in this table, even though they can be used in computations.

Table 3-4. Special IEEE 754 quantities

Special quantity	64-bit IEEE 754 representation
Positive infinity	`0x7ff0000000000000L`
Negative infinity	`0xfff0000000000000L`

Special quantity	64-bit IEEE 754 representation
Not a number (NaN)	`0x7ff0000000000001L` through `0x7fffffffffffffffL` and `0xfff0000000000001L` through `0xffffffffffffffffL`
Negative zero	`0x8000000000000000`
Positive zero	`0x0000000000000000`

These special quantities can result from computations that go outside the acceptable bounds. The expression `1/0.0` in Java computes to be the quantity positive infinity. If the statement had instead read `double x=1/0`, then the Java virtual machine would throw an `ArithmeticException` since this expression computes the integer division of two numbers.

Example Algorithm

To illustrate our algorithm template, we now describe the **Graham's Scan** algorithm for computing the convex hull for a collection of points. This was the problem presented in Chapter 1 and illustrated in Figure 1-3.

Name and Synopsis

Graham's Scan computes the convex hull for a collection of Cartesian points. It locates the lowest point, *low*, in the input set *P* and sorts the remaining points { *P* – *low* } in *reverse* polar angle with respect to the lowest point. With this order in place, the algorithm can traverse *P* clockwise from its lowest point. Every left turn of the last three points in the hull being constructed reveals that the last hull point was incorrectly chosen so it can be removed.

Input/Output

A convex hull problem instance is defined by a collection of points, *P*.

The output will be a sequence of (*x*, *y*) points representing a clockwise traversal of the convex hull. It shouldn't matter which point is first.

Context

This algorithm is suitable for Cartesian points. If the points, for example, use a different coordinate system where increasing *y* values reflect lower points in the plane, then the algorithm should compute *low* accordingly. Sorting the points by polar angle requires trigonometric calculations.

Graham's Scan Summary

Best, Average, Worst: O(*n log n*)

```
graham(P)
  low = point with lowest y coordinate in P ❶
  remove low from P
  sort P by descending polar angle with respect to low ❷

  hull = {P[n-2], low} ❸
  for i = 0 to n-1 do
    while (isLeftTurn(secondLast(hull), last(hull), P[i])) do
      remove last point in hull ❹

    add P[i] to hull

  remove duplicate last point ❺
  return hull
```

❶ Ties are broken by selecting the point with lowest *x* coordinate.

❷ P[0] has max polar angle and P[n – 2] has min polar angle.

❸ Form hull clockwise starting with min polar angle and `low`.

❹ Every turn to the left reveals last hull point must be removed.

❺ Because it will be P[n – 2].

Solution

If you solve this problem by hand, you probably have no trouble tracing the appropriate edges, but you might find it hard to explain the exact sequence of steps you took. The key step in this algorithm is sorting the points by descending polar angle with respect to the lowest point in the set. Once ordered, the algorithm proceeds to "walk" along these points, extending a partially constructed hull and adjusting its structure if the last three points of the hull ever form a left turn, which would indicate a nonconvex shape. See Example 3-1.

Example 3-1. GrahamScan implementation

```
public class NativeGrahamScan implements IConvexHull {
  public IPoint[] compute (IPoint[] pts) {
    int n = pts.length;
    if (n < 3) { return pts; }

    // Find lowest point and swap with last one in points[] array,
    // if it isn't there already
```

```
int lowest = 0;
double lowestY = pts[0].getY();
for (int i = 1; i < n; i++) {
  if (pts[i].getY() < lowestY) {
    lowestY = pts[i].getY();
    lowest = i;
  }
}

if (lowest != n-1) {
  IPoint temp = pts[n-1];
  pts[n-1] = pts[lowest];
  pts[lowest] = temp;
}

// sort points[0..n-2] by descending polar angle with respect
// to lowest point points[n-1].
new HeapSort<IPoint>().sort(pts, 0, n-2,
                            new ReversePolarSorter(pts[n-1]));

// three points *known* to be on the hull are (in this order) the
// point with lowest polar angle (points[n-2]), the lowest point
// (points[n-1]), and the point with the highest polar angle
// (points[0]). Start with first two
DoubleLinkedList<IPoint> list = new DoubleLinkedList<IPoint>();
list.insert(pts[n-2]);
list.insert(pts[n-1]);

// If all points are collinear, handle now to avoid worrying about later
double firstAngle = Math.atan2(pts[0].getY() - lowest,
                               pts[0].getX() - pts[n-1].getX());
double lastAngle = Math.atan2(pts[n-2].getY() - lowest,
                              pts[n-2].getX() - pts[n-1].getX());
if (firstAngle == lastAngle) {
  return new IPoint[] { pts[n-1], pts[0] };
}

// Sequentially visit each point in order, removing points upon
// making mistake. Because we always have at least one "right
// turn," the inner while loop will always terminate
for (int i = 0; i < n-1; i++) {
  while (isLeftTurn(list.last().prev().value(),
                    list.last().value(),
                    pts[i])) {
    list.removeLast();
  }

  // advance and insert next hull point into proper position
  list.insert(pts[i]);
}

// The final point is duplicated, so we take n-1 points starting
```

```
    // from lowest point.
    IPoint hull[] = new IPoint[list.size()-1];
    DoubleNode<IPoint> ptr = list.first().next();
    int idx = 0;
    while (idx < hull.length) {
      hull[idx++] = ptr.value();
      ptr = ptr.next();
    }

    return hull;
  }

  /** Use Collinear check to determine left turn. */
  public static boolean isLeftTurn(IPoint p1, IPoint p2, IPoint p3) {
    return (p2.getX() - p1.getX())*(p3.getY() - p1.getY()) -
           (p2.getY() - p1.getY())*(p3.getX() - p1.getX()) > 0;
  }
}

/** Sorter class for reverse polar angle with respect to a given point. */
class ReversePolarSorter implements Comparator<IPoint> {
  /** Stored x,y coordinate of base point used for comparison. */
  final double baseX;
  final double baseY;

  /** PolarSorter evaluates all points compared to base point. */
  public ReversePolarSorter(IPoint base) {
    this.baseX = base.getX();
    this.baseY = base.getY();
  }

  public int compare(IPoint one, IPoint two) {
    if (one == two) { return 0; }

    // make sure both have computed angle using atan2 function.
    // Works because one.y is always larger than or equal to base.y
    double oneY = one.getY();
    double twoY = two.getY();
    double oneAngle = Math.atan2(oneY - baseY, one.getX() - baseX);
    double twoAngle = Math.atan2(twoY - baseY, two.getX() - baseX);

    if (oneAngle > twoAngle) { return -1; }
    else if (oneAngle < twoAngle) { return +1; }

    // if same angle, then must order by decreasing magnitude
    // to ensure that the convex hull algorithm is correct
    if (oneY > twoY) { return -1; }
    else if (oneY < twoY) { return +1; }

    return 0;
  }
}
```

If all $n > 2$ points are collinear then in this special case, the hull consists of the two extreme points in the set. The computed convex hull might contain multiple consecutive points that are collinear because no attempt is made to remove them.

Analysis

Sorting n points requires $O(n \log n)$ performance, as described in Chapter 4. The rest of the algorithm has a `for` loop that executes n times, but how many times does its inner `while` loop execute? As long as there is a left turn, a point is removed from the hull, until only the first three points remain. Since no more than n points are added to the hull, the inner `while` loop can execute no more than n times in total. Thus, the performance of the `for` loop is $O(n)$. The result is that the overall algorithm performance is $O(n \log n)$ since the sorting costs dominates the cost of the whole computation.

Common Approaches

This section presents the fundamental algorithm approaches used in the book. You need to understand these general strategies for solving problems so you see how they can be applied to solve specific problems. Chapter 10 contains additional strategies, such as seeking an acceptable approximate answer rather than the definitive one, or using randomization with a large number of trials to converge on the proper result rather than using an exhaustive search.

Greedy

A Greedy strategy completes a task of size n by incrementally solving the problem in steps. At each step, a Greedy algorithm will make the best local decision it can given the available information, typically reducing the size of the problem being solved by one. Once all n steps are completed, the algorithm returns the computed solution.

To sort an array A of n numbers, for example, the Greedy **Selection Sort** algorithm locates the largest value in $A[0, n - 1]$ and swaps it with the element in location $A[n - 1]$, which ensures $A[n - 1]$ is in its proper location. Then it repeats the process to find the largest value remaining in $A[0, n - 2]$, which is similarly swapped with the element in location $A[n - 2]$. This process continues until the entire array is sorted. For more detail, see Chapter 4.

You can identify a Greedy strategy by the way that subproblems being solved shrink very slowly as an algorithm processes the input. When a subproblem can be completed in $O(\log n)$ then a Greedy strategy will exhibit $O(n \log n)$ performance. If the subproblem requires $O(n)$ behavior, as it does here with **Selection Sort**, then the overall performance will be $O(n^2)$.

Divide and Conquer

A Divide and Conquer strategy solves a problem of size *n* by dividing it into two independent subproblems, each about half the size of the original problem. Quite often the solution is recursive, terminating with a base case that can be solved trivially. There must be some *resolution* computation that can determine the solution for a problem when given two solutions for two smaller subproblems.

To find the largest element in an array of *n* numbers, for example, the recursive function in Example 3-2 constructs two subproblems. Naturally, the maximum element of the original problem is simply the larger of the maximum values of the two subproblems. Observe how the recursion terminates when the size of the subproblem is 1, in which case the single element *vals*[*left*] is returned.

Example 3-2. Recursive Divide and Conquer approach to finding maximum element in array

```
/** Invoke the recursion. */
public static int maxElement (int[] vals) {
  if (vals.length == 0) {
    throw new NoSuchElementException("No Max Element in Empty Array.");
  }
  return maxElement(vals, 0, vals.length);
}

/** Compute maximum element in subproblem vals[left, right).
 * Note that the right endpoint is not part of the range. */
static int maxElement (int[] vals, int left, int right) {
  if (right - left == 1) {
    return vals[left];
  }

  // compute subproblems
  int mid = (left + right)/2;
  int max1 = maxElement(vals, left, mid);
  int max2 = maxElement(vals, mid, right);

  // Resolution: compute result from results of subproblems
  if (max1 > max2) { return max1; }
  return max2;
}
```

A Divide and Conquer algorithm structured as shown in Example 3-2 will exhibit O(*n*) performance if the *resolution* step can be accomplished in constant O(*1*) time, as it does here. When the resolution step itself requires O(*n*) computations, then the overall performance will be O(*n log n*). Note that you can more rapidly find the largest element in a collection by scanning each element and storing the largest one found. Let this be a brief reminder that Divide and Conquer will not always provide the fastest implementation.

Dynamic Programming

Dynamic Programming is a variation on Divide and Conquer that solves a problem by subdividing it into a number of simpler subproblems that are solved in a specific order. It solves each smaller problem just once and stores the results for future use to avoid unnecessary recomputation. It then solves problems of increasing size, *composing* together solutions from the results of these smaller subproblems. In many cases, the computed solution is provably optimal for the problem being solved.

Dynamic Programming is frequently used for optimization problems where the goal is to minimize or maximize a particular computation. The best way to explain Dynamic Programming is to show a working example.

Scientists often compare DNA sequences to determine their similarities. If you represent such a DNA sequence as a string of characters—A, C, T, or G—then the problem is restated as computing the *minimum edit distance* between two strings. That is, given a base string s_1 and a target string s_2 determine the fewest number of edit operations that transform s_1 into s_2 if you can:

- Replace a character in s_1 with a different character
- Remove a character in s_1
- Insert a character into s_1

For example, given a base string, s_1, representing the DNA sequence "GCTAC" you only need three edit operations to convert this to the target string, s_2, whose value is "CTCA":

- Replace the fourth character ("A") with a "C"
- Remove the first character ("G")
- Replace the last "C" character with an "A"

This is not the only such sequence of operations, but you need at least three edit operations to convert s_1 to s_2. For starters, the goal is to compute the *value* of the optimum answer—i.e., the number of edit operations—rather than the actual sequence of operations.

Dynamic Programming works by storing the results of simpler subproblems; in this example, you can use a two-dimensional matrix, $m[i][j]$, to record the result of computing the minimum edit distance between the first i characters of s_1 and the first j characters of s_2. Start by constructing the following initial matrix:

```
0  1  2  3  4
1  .  .  .  .
2  .  .  .  .
```

3 . . . .

4 . . . .

5 . . . .

In this table, each row is indexed by *i* and each column is indexed by *j*. Upon completion, the entry *m*[0][4] (the top-right corner of the table) will contain the result of the edit distance between the first 0 characters of s_1 (i.e., the empty string "") and the first four characters of s_2 (i.e., the whole string "CTCA"). The value of *m*[0][4] is 4 because you have to insert four characters to the empty string to equal s_2. Similarly, *m*[3][0] is 3 because starting from the first three characters of s_1 (i.e., "GCT") you have to delete three characters to equal the first zero characters of s_2 (i.e., the empty string "").

The trick in Dynamic Programming is an optimization loop that shows how to compose the results of these subproblems to solve larger ones. Consider the value of *m*[1][1], which represents the edit distance between the first character of s_1 ("G") and the first character of s_2 ("C"). There are three choices:

- Replace the "G" character with a "C" for a cost of 1
- Remove the "G" and insert the "C" for a cost of 2
- Insert a "C" character and then delete the "G" character for a cost of 2

You clearly want to record the minimum cost of each of these three choices, so *m*[1][1] = 1. How can you generalize this decision? Consider the computation shown in Figure 3-2.

These three options for computing *m*[*i*][*j*] represent the following:

Replace cost
: Compute the edit distance between the first $i - 1$ characters of s_1 and the first $j - 1$ characters of s_2 and then add 1 for replacing the j^{th} character of s_2 with the i^{th} character of s_1, if they are different.

Remove cost
: Compute the edit distance between the first $i - 1$ characters of s_1 and the first j characters of s_2 and then add 1 for removing the i^{th} character of s_1.

Insert cost
: Compute the edit distance between the first i characters of s_1 and the first $j - 1$ characters of s_2 and then add 1 for inserting the j^{th} character of s_2.

Visualizing this computation, you should see that Dynamic Programming must evaluate the subproblems in the proper order (i.e., from top row to bottom row, and left to right within each row, as shown in Example 3-3). The computation proceeds from row index value $i = 1$ to $len(s_1)$. Once the matrix *m* is populated with its initial

values, a nested `for` loop computes the minimum value for each of the subproblems in order until all values in *m* are computed. This process is not recursive, but rather, it uses results of past computations for smaller problems. The result of the full problem is found in $m[len(s_1)][len(s_2)]$.

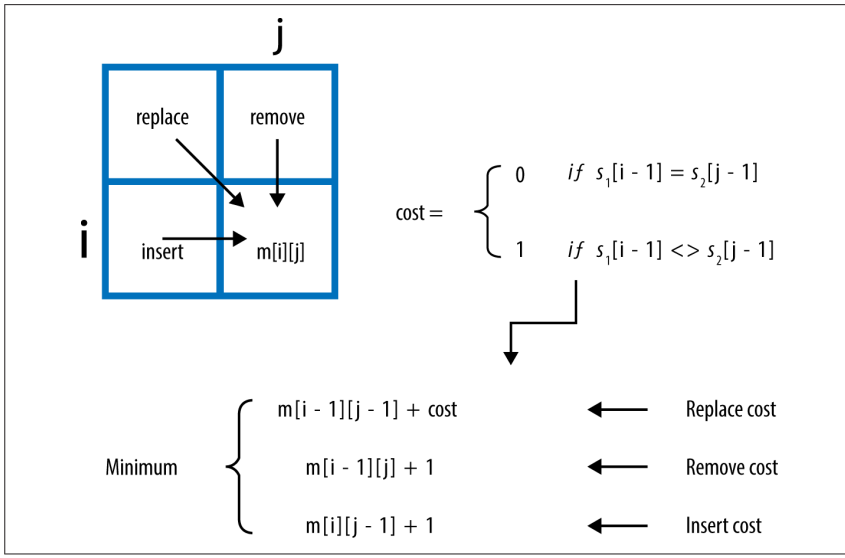

Figure 3-2. Computing m[i][j]

Example 3-3. Minimum edit distance solved using Dynamic Programming

```
def minEditDistance(s1, s2):
  """Compute minimum edit distance converting s1 -> s2."""
  len1 = len(s1)
  len2 = len(s2)

  # Create two-dimensional structure such that m[i][j] = 0
  # for i in 0 .. len1 and for j in 0 .. len2
  m = [None] * (len1 + 1)
  for i in range(len1+1):
    m[i] = [0] * (len2+1)

  # set up initial costs on horizontal and vertical
  for i in range(1, len1+1):
    m[i][0] = i
  for j in range(1, len2+1):
    m[0][j] = j

  # compute best
  for i in range(1,len1+1):
    for j in range(1,len2+1):
      cost = 1
```

```
      if s1[i-1] == s2[j-1]: cost = 0

      replaceCost = m[i-1][j-1] + cost
      removeCost  = m[i-1][j] + 1
      insertCost  = m[i][j-1] + 1
      m[i][j]     = min(replaceCost,removeCost,insertCost)

  return m[len1][len2]
```

Table 3-5 shows the final value of *m*.

Table 3-5. Result of all subproblems

0	1	2	3	4
1	1	2	3	4
2	1	2	2	3
3	2	1	2	3
4	3	2	2	2
5	4	3	2	3

The cost of subproblem *m*[3][2] = 1 which is the edit distance of the string "GCT" and "CT". As you can see, you only need to delete the first character which validates this cost is correct. This code only shows how to compute the minimum edit distance; to actually record the sequence of operations that would be performed, a *prev*[*i*][*j*] matrix records which of the three cases was selected when computing the minimum value of *m*[*i*][*j*]. To recover the operations, trace backwards from m[*len*(s_1)][*len*(s_2)] using decisions recorded in *prev*[*i*][*j*] stopping once *m*[0][0] is reached. This revised implementation is shown in Example 3-4.

Example 3-4. Minimum edit distance with operations solved using Dynamic Programming

```
REPLACE = 0
REMOVE  = 1
INSERT  = 2

def minEditDistance(s1, s2):
  """Compute minimum edit distance converting s1 -> s2 with operations."""
  len1 = len(s1)
  len2 = len(s2)

  # Create two-dimensional structure such that m[i][j] = 0
  # for i in 0 .. len1 and for j in 0 .. len2
  m = [None] * (len1 + 1)
  op = [None] * (len1 + 1)
  for i in range(len1+1):
```

```
    m[i] = [0] * (len2+1)
    op[i] = [-1] * (len2+1)

  # set up initial costs on horizontal and vertical
  for j in range(1, len2+1):
    m[0][j] = j
  for i in range(1, len1+1):
    m[i][0] = i

  # compute best
  for i in range(1,len1+1):
    for j in range(1,len2+1):
      cost = 1
      if s1[i-1] == s2[j-1]: cost = 0

      replaceCost = m[i-1][j-1] + cost
      removeCost  = m[i-1][j] + 1
      insertCost  = m[i][j-1] + 1
      costs       = [replaceCost,removeCost,insertCost]
      m[i][j]     = min(costs)
      op[i][j]    = costs.index(m[i][j])

  ops = []
  i = len1
  j = len2
  while i != 0 or j != 0:
    if op[i][j] == REMOVE or j == 0:
      ops.append('remove {}-th char {} of {}'.format(i,s1[i-1],s1))
      i = i-1
    elif op[i][j] == INSERT or i == 0:
      ops.append('insert {}-th char {} of {}'.format(j,s2[j-1],s2))
      j = j-1
    else:
      if m[i-1][j-1] < m[i][j]:
        fmt='replace {}-th char of {} ({}) with {}'
        ops.append(fmt.format(i,s1,s1[i-1],s2[j-1]))
      i,j = i-1,j-1

  return m[len1][len2], ops
```

References

Goldberg, D., "What Every Computer Scientist Should Know About Floating-Point Arithmetic," *ACM Computing Surveys*, March 1991, *http://docs.sun.com/source/806-3568/ncg_goldberg.html*.

Venners, B., "Floating-point arithmetic: A look at the floating-point support of the Java virtual machine," *JavaWorld*, 1996, *http://www.javaworld.com/article/2077257/learn-java/floating-point-arithmetic.html*.